



BSc (Hons) in **Information Technology**

Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{ 'wall_ahead': True/False, 'food_here': True/False }` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent` .
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)` , first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A Table-Driven Agent is impractical for complex environments because it would need to store an action for every possible percept sequence. As the agent's lifetime increases, the number of possible percept sequences grows exponentially. This results in a combinatorial explosion, causing the required table to become extremely large and impossible to construct and maintain in practice.

For a game such as Chess, the number of possible game states and percept histories is enormous. Therefore, storing a separate action for every possible history would require an unrealistic amount of memory and computation. Instead of using a complete table, intelligent agents use Agent Programs that make decisions based on rules, models, or other reasoning mechanisms.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The SimpleReflexAgent uses IF-THEN condition-action rules to select an action based only on the current percept.

The first rule is:

```
if percept['food_here']:
    return 'Up'
```

This represents:

IF food_here THEN move forward.

The second rule is:

```
if percept['wall_ahead']:
    return 'Left'
```

This represents:

IF wall_ahead THEN turn left.

Finally:

```
return 'Up'
```

represents the default ELSE condition:

ELSE move forward.

These rules demonstrate the Simple Reflex Agent architecture because the agent selects its action directly from the current percept without using information about previous percepts or actions.



3. (Analyze) Your SimpleReflexAgent likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The Simple Reflex Agent can become trapped in an infinite loop because it operates using only the information available in its current percept. In the partially observable environment, the agent cannot see its exact global position or remember where it has previously travelled.

For example, when the agent encounters a wall, its rule may always instruct it to turn left. If the same situation occurs again, the agent has no memory that it has already tried this action. Therefore, it makes the same decision again.

The combination of partial observability and the absence of percept history prevents the agent from recognizing that it is repeating a previous sequence of actions. This can cause the agent to repeatedly move around the same area or become trapped inside a corner or U-shaped wall.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The `ModelBasedAgent` maintains an internal state using variables such as `visited_states`, `previous_percept`, and `previous_action`.

The Sensor Model is represented by the percept received from the environment:

```
percept['wall_ahead']  
percept['food_here']
```

These values describe what the agent can currently observe from its limited sensors.

The agent updates its internal state by storing the current percept and previous action:

```
self.previous_percept = percept.copy()  
self.previous_action = action
```

The Transition Model is represented by the agent using its previous action and current percept to determine how its internal situation has changed. For example, if the agent previously chose Left when facing a wall and encounters the same situation again, it can choose Right instead.

Therefore, unlike the Simple Reflex Agent, the Model-Based Agent uses information from previous interactions when selecting its next action. This internal state allows it to avoid repeatedly applying the same rule and provides a mechanism for escaping loops.

Finalize and Lock Lab 02 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING